

Источники данных по категориям

SourceCraft Repo Health Score · какие запросы API нужны для каждой категории · 20.09.2026

Documentation

Что нужно	Запрос
Наличие README, LICENSE, CONTRIBUTING, CODEOWNERS, структура проекта	<code>GET /repos/{org}/{repo}/trees</code> (рекурсивно)
Содержимое README — качество, инструкции запуска / сборки / тестов	RAW-файл или клон. Дерево отдаёт только имена файлов

CI/CD

Что нужно	Запрос
Наличие CI, статусы, success rate, длительность, стабильность	<code>GET /repos/{org}/{repo}/ci/cd/runs</code>
Наличие конфига <code>.sourcecraft/ci.yaml</code>	из того же дерева файлов

Считать по `status` и `dates.started_at` / `dates.finished_at`. Незавершённые прогоны (`created`, `prepared`, `processing`) в знаменатель не брать. Разделять по `event_type`: падение на push в основную ветку и падение на PR — это разное здоровье.

Security

Что нужно	Запрос
Статус скана, сколько уязвимостей осталось открытыми, тренд	<code>GET /v1/scans/latest?gitRepo={id}</code>
Детали находок: файл, строка, правило — для рекомендаций	<code>GET /v1/findings?gitRepo={id}&pageSize=...&pageToken=</code>
Статусы исправления по группам дефектов	<code>GET /v1/defect-groups</code>
Какие анализаторы существуют	<code>GET /v1/defect-groups/types</code>

Другой домен: `appsec.sourcecraft.tech`. Для Score хватит одного `scans/latest`, остальное — только при открытии страницы репозитория. `pageSize` и `pageToken` обязательны, максимум 250.

Activity

Что нужно	Запрос
Контрибьюторы	<code>GET /repos/{org}/{repo}/contributors</code>
Pull requests	<code>GET /repos/{org}/{repo}/pulls</code>
Релизы	<code>GET /repos/{org}/{repo}/releases</code>
Теги, ветки	<code>GET /repos/{org}/{repo}/tags</code> , <code>/branches</code>
Дата последнего обновления	из объекта репозитория, поле <code>last_updated</code>
Количество и частота коммитов, распределение по авторам	Клон. <code>git log</code> — метода со списком коммитов в API нет

Issues

Что нужно	Запрос
Открытые / закрытые, зависшие, время до закрытия	<code>GET /repos/{org}/{repo}/issues</code>
Время до первого ответа	комментарии задачи — по <code>author</code> и <code>created_at</code>

Комментарии тянутся по каждой задаче отдельно, поэтому ограничьте выборку последними N задачами — иначе запросов будет слишком много.

Code health

Что нужно	Запрос
TODO / FIXME, количество и плотность	Клон. <code>grep</code> по исходникам
Возраст TODO / FIXME	Клон. <code>git blame</code> по найденным строкам
Объём кода для нормализации плотности	Клон. подсчёт строк (<code>tokei</code> / <code>scc</code>)

Получение самих репозиториев

Сценарий	Запрос	Токен
Публичный рейтинг	<code>DiscoverRepositories</code>	сервисный PAT команды
Репозитории пользователя	<code>GET /orgs/{org_slug}/repos</code>	PAT пользователя
Один репозиторий по ссылке	<code>GET /repos/{org}/{repo}</code>	PAT пользователя

Везде крутить `next_page_token` , пока не пропадёт. Иначе получите только первую страницу.

Про клонирование

Клон нужен целиком для **Code health**, наполовину для **Activity** и для содержимого README в **Documentation**. Клонировать все репозитории рейтинга дорого, поэтому сбор делится на два уровня.

Быстрый проход — без клона

Documentation по именам файлов, CI/CD, Security, Issues, Activity по API. Четыре с половиной категории из шести. Годится для массового пересчёта всего каталога по расписанию.

Полный проход — с клоном

Добавляется Code health и коммиты. Запускается для топа рейтинга и когда пользователь сам просит проанализировать репозиторий.

- Клонировать мелко: `git clone --filter=blob:none` или `--depth`
- Рабочую копию удалять сразу после разбора — ТЗ запрещает хранить исходники дольше необходимого
- `git blame` не гонять по всему репозиторию: сначала `grep`, потом `blame` только по найденным строкам, с лимитом на число файлов